

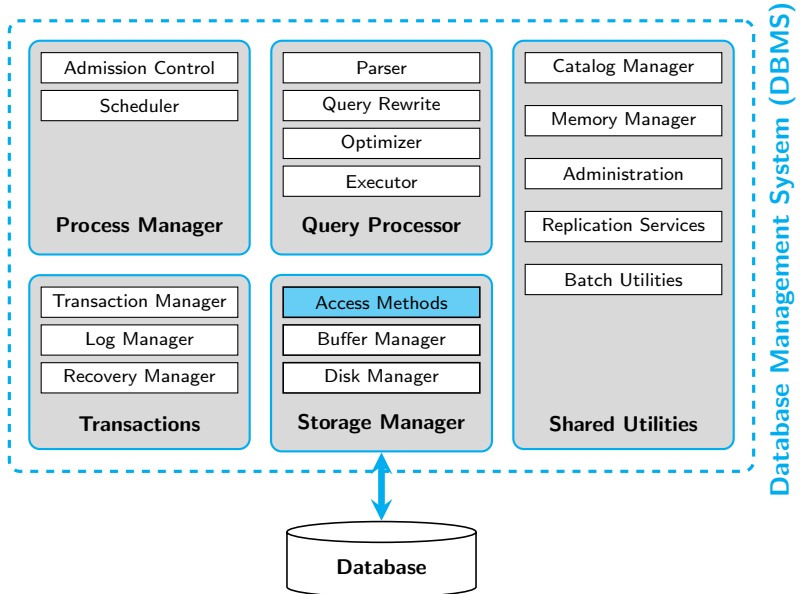
Indices: Hashing

Clase 15

IIC2413 - Sección 3

Prof. Cristian Riveros

Índices basados en Hashing



Índices basados en Hashing

- Eficientes para consultas de **valores** (value query):

```
SELECT  *  
FROM    Players  
WHERE   age = 30
```

- Una alternativa a B+tree.
- Soportado en la mayoría de los motores de BD.

Outline

Static Hashing

Extendable Hashing

Sobre índices

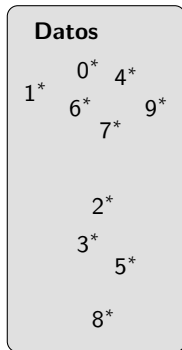
Outline

Static Hashing

Extendable Hashing

Sobre índices

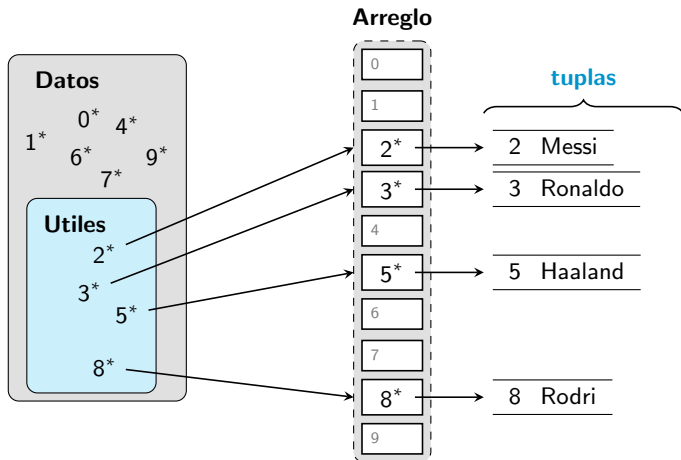
Motivación: arreglo como índice



Arreglo



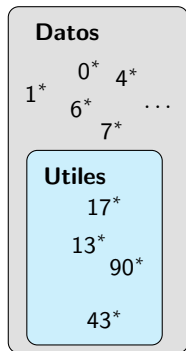
Motivación: arreglo como índice



¿cuánto **tiempo** nos demoramos en **buscar o insertar** una tupla?

$\mathcal{O}(1)$, menor tiempo posible

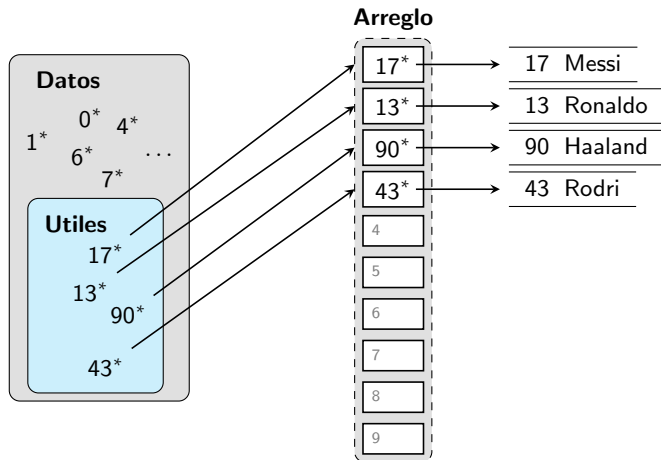
Motivación: universo infinito



Arreglo



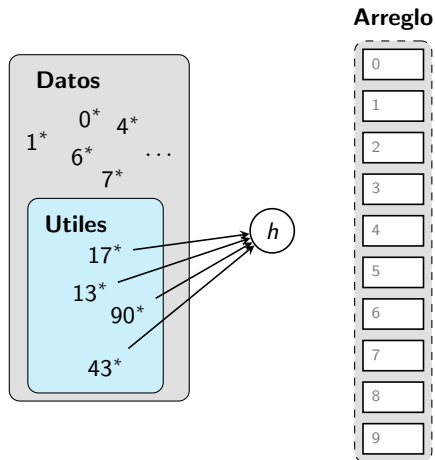
Motivación: universo infinito



¿cuánto **tiempo** nos demoramos en **buscar** una tupla?

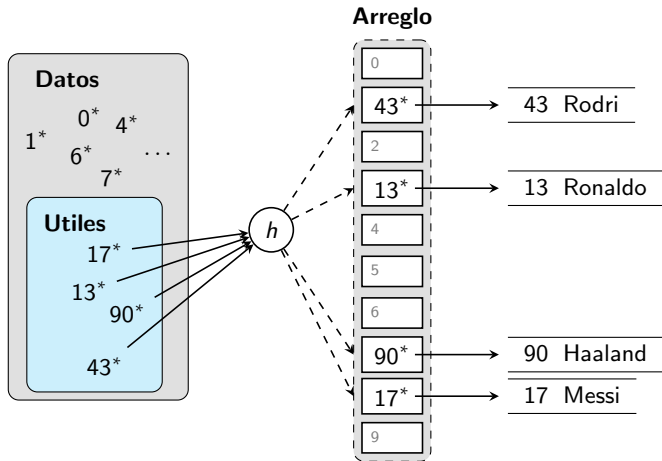
$\mathcal{O}(n)$, mayor tiempo posible

Motivación: uso de función h



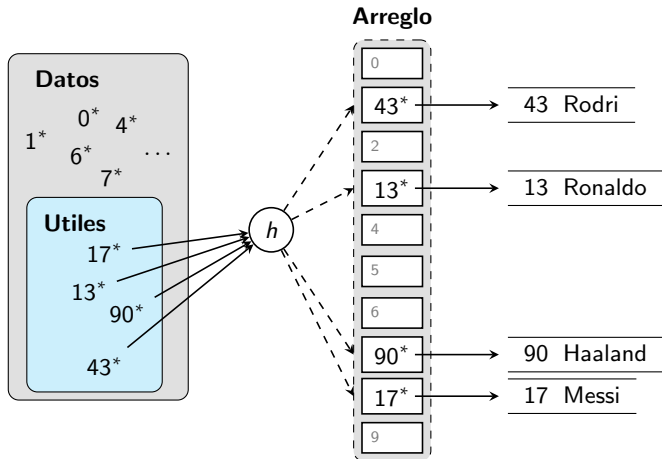
$$h: \mathbb{N} \rightarrow \{0, \dots, 9\}$$

Motivación: uso de función h



$$h: \mathbb{N} \rightarrow \{0, \dots, 9\}$$

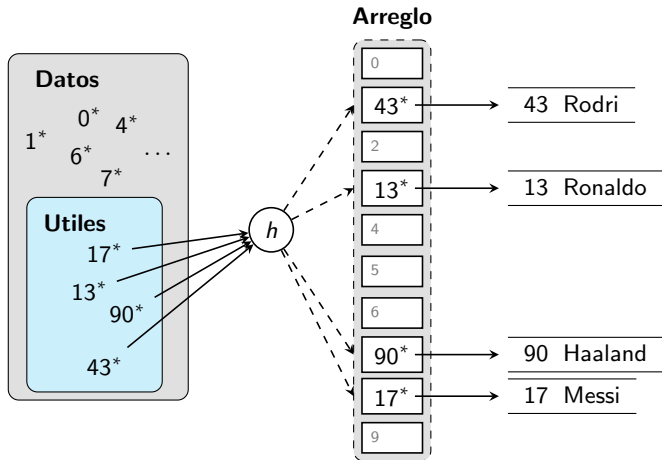
Motivación: uso de función h



¿cuánto **tiempo** nos demoramos en **buscar** una tupla?

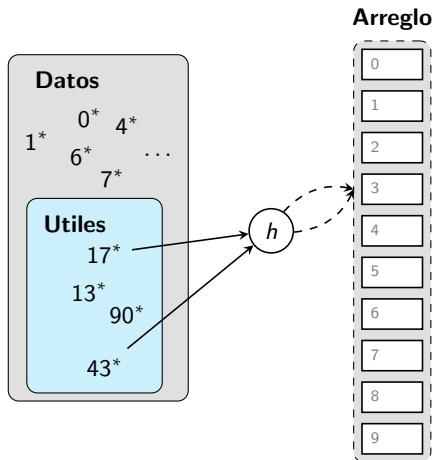
$\mathcal{O}(1)$, menor tiempo posible

Motivación: uso de función h



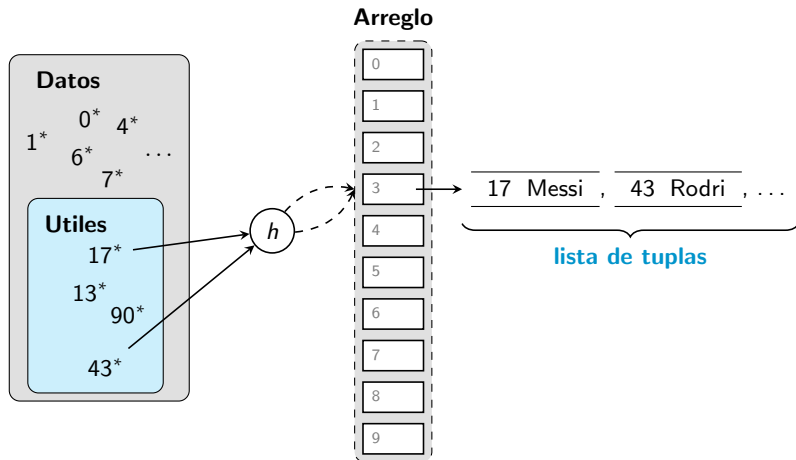
¿qué **problema** podemos tener al usar una función h ?

Motivación: colisiones de función h



¿qué hacemos si ocurre una **colisión**?

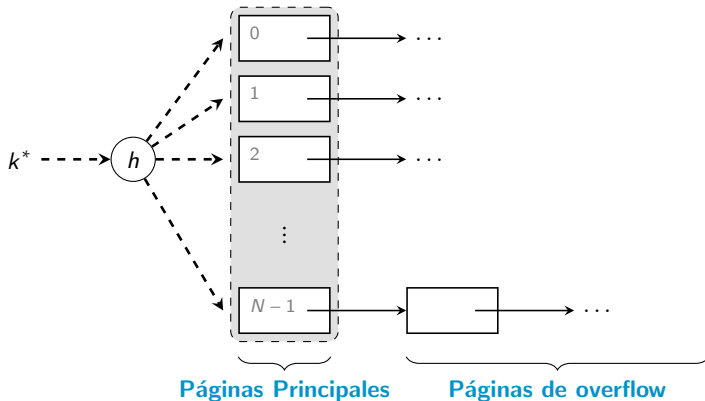
Motivación: solución a colisiones



¿cuánto **tiempo** nos demoramos en **buscar o insertar** una tupla?

(largo de la lista)

Índice de static hashing



Usamos una **función de hash** h tal que:

$$h: \text{datatype} \rightarrow \{0, \dots, N-1\}$$

Operaciones en static hashing

Para las operaciones $\text{search}(k^*)$, $\text{insert}(k^*)$, $\text{delete}(k^*)$:

1. Computar el valor $h(k^*)$.
2. Acceder al **bucket** en la posición $h(k^*)$.
3. Buscar, insertar, o eliminar la tupla en la página $h(k^*)$ o en sus **páginas de overflow**.

Costo de cada operación: $1 + \#(\text{páginas de overflow})$

¿cómo disminuimos el número de colisiones?

Problema

Colisión: valores k_1 y k_2 tal que $h(k_1) = h(k_2)$.

Estrategia

Encontrar una función h , tal que, para toda secuencia de valores:

$$k_1, k_2, \dots, k_n$$

los valores $h(k_1), \dots, h(k_n)$ **parecen aleatorios**.

Funciones de Hash

OJO: las funciones de Hash NO son aleatorias
(pero parecen aleatorias)

Ejemplos simples de funciones de Hash

- División:

$$h(k^*) = (a * k + b) \bmod N$$

- Multiplicación:

$$\lfloor N \cdot \underbrace{(Q \cdot k - \lfloor Q \cdot k \rfloor)}_{\text{parte decimal de } Q \cdot k} \rfloor$$

Para cualquier $Q \in \mathbb{Q}$, como por ejemplo:

$$Q = \frac{2}{\sqrt{5} + 1}$$

(inverso del golden ratio).

Para más información sobre **funciones de Hash**
véase el curso de Estructura de Datos o Criptografía.

Costo de operaciones en Hash Index

Dado:

- una buena selección de la función de hash h y
- $\#tuplas \propto B \cdot N$
con B el tamaño de una página y N el número de buckets.

se tiene que el costo de cada operación es **constante**.

$$\text{Costo de cada operación} \approx \frac{\#tuplas}{B \cdot N}$$

¿cuál es el **problema** de este costo?

Outline

Static Hashing

Extendable Hashing

Sobre índices

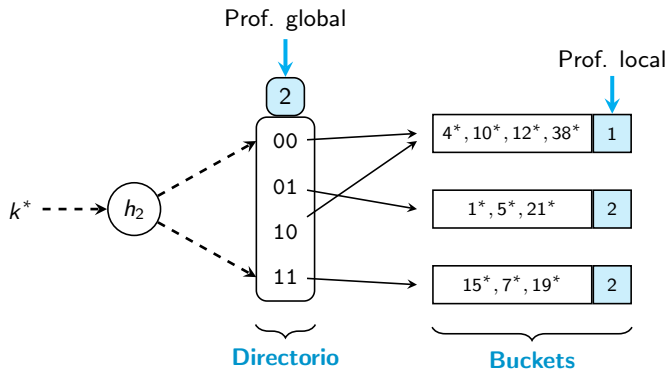
Extendable Hashing

*Aumentar el número de buckets
dependiendo del número de tuplas.*

Estrategia

1. Usar un **directorio** de buckets principales.
2. **Duplicar** el tamaño del direc. cuando haya un **overflow** de un bucket.

Extendable Hashing



h_n considera los **últimos n bits** de k^*

Profundidad global y profundidad local

Profundidad **global** n :

- La función h_n considera los n -últimos bits.
- El directorio es de tamaño 2^n .

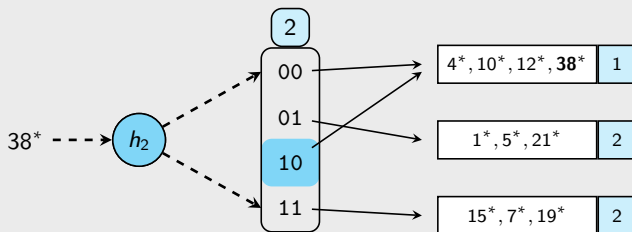
Profundidad **local** d para cada bucket:

- $h_d(k_1^*) = h_d(k_2^*)$ para todo k_1^*, k_2^* en el mismo bucket.
- Todos los elementos del bucket coinciden en los últimos d bits.

Búsqueda en Extendable Hashing

Ejemplo: Buscar 38^*

$$38^* = 100110 \quad h_2(38^*) = 10$$



Insertar en Extendable Hashing

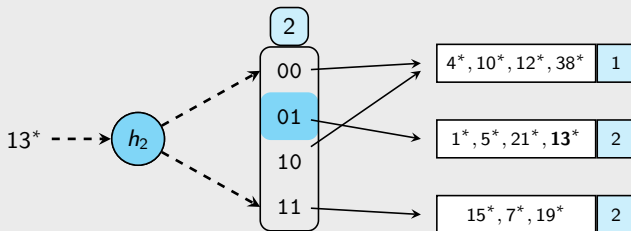
Insertar dato k^* con profundidad global n

1. Computamos $h_n(k^*)$ (los últimos n bits de k^*).
2. Buscamos en el directorio el puntero p para la entrada $h_n(k^*)$.
3. Seguimos el puntero p al bucket B correspondiente.
4. Si hay espacio en B , insertamos k^* en B .

Insertar en Extendable Hashing

Ejemplo: insertar $k^* = 13^*$

$$13^* = 1101 \quad h_2(13^*) = 01$$



Insertar en Extendable Hashing

Insertar dato k^* con profundidad global n

1. Computamos $h_n(k^*)$ (los últimos n bits de k^*).
2. Buscamos en el directorio el puntero p para la entrada $h_n(k^*)$.
3. Seguimos el puntero p al bucket B correspondiente.
4. Si hay espacio en B , insertamos k^* en B .
5. Si **NO hay espacio** en B con profundidad local d :
 - Hacemos un **bucket split** de B .

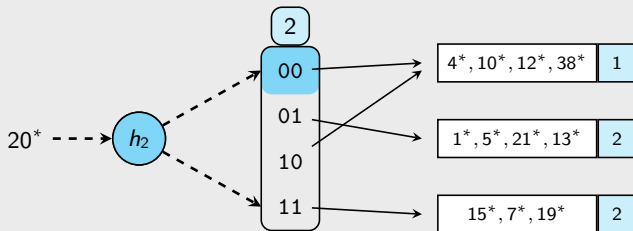
Insertar en Extendable Hashing: Bucket Split

Bucket split

Dado un bucket B con profundidad local d ,
dividimos sus elementos en dos buckets según el bit $d + 1$.

Ejemplo: Insertar $k^* = 20^*$

$$20^* = 10100 \quad h_2(20^*) = 00$$

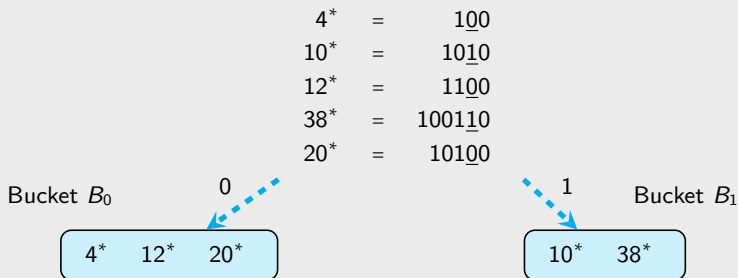


Insertar en Extendable Hashing: Bucket Split

Bucket split

Dado un bucket B con profundidad local d ,
dividimos sus elementos en dos buckets según el bit $d + 1$.

Ejemplo: Insertar $k^* = 20^*$



Insertar en Extendable Hashing

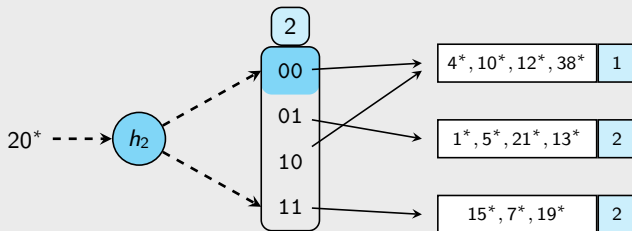
Insertar dato k^* con profundidad global n

1. Computamos $h_n(k^*)$ (los últimos n bits de k^*).
2. Buscamos en el directorio el puntero p para la entrada $h_n(k^*)$.
3. Seguimos el puntero p al bucket B correspondiente.
4. Si hay espacio en B , insertamos k^* en B .
5. Si NO hay espacio en B con profundidad local d :
 - Hacemos un bucket split de B .
 - **Aumentamos** la profundidad local de B_0 y B_1 a $d + 1$.
 - Si $n > d$, **insertamos** B_0 y B_1 en el directorio.

Insertar en Extendable Hashing: insertar en el directorio

Ejemplo: Insertar $k^* = 20^*$

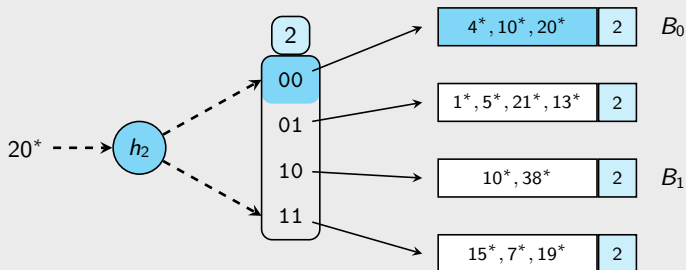
$$20^* = 10100 \quad h_2(20^*) = 00$$



Insertar en Extendable Hashing: insertar en el directorio

Ejemplo: Insertar $k^* = 20^*$

$$20^* = 10100 \quad h_2(20^*) = 00$$



Insertar en Extendable Hashing

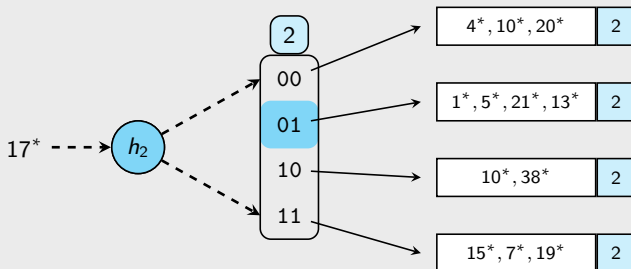
Insertar dato k^* con profundidad global n

1. Computamos $h_n(k^*)$ (los últimos n bits de k^*).
2. Buscamos en el directorio el puntero p para la entrada $h_n(k^*)$.
3. Seguimos el puntero p al bucket B correspondiente.
4. Si hay espacio en B , insertamos k^* en B .
5. Si NO hay espacio en B con profundidad local d :
 - Hacemos un bucket split de B .
 - Aumentamos la profundidad local de B_0 y B_1 a $d + 1$.
 - Si $n > d$, insertamos B_0 y B_1 en el directorio.
 - Si $n = d$, hacemos un **doblamiento del directorio**.

Insertar en Extendable Hashing: doblamiento del directorio

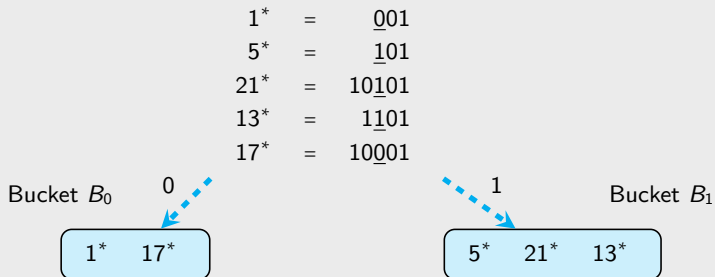
Ejemplo: insertar $k^* = 17^*$

$$17^* = 10001 \quad h_2(17^*) = 01$$



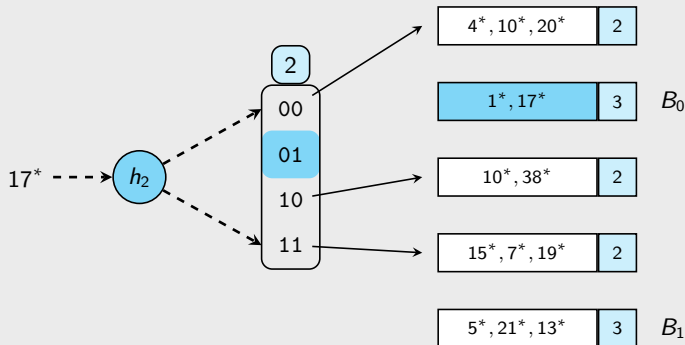
Insertar en Extendable Hashing: doblamiento del directorio

Ejemplo: insertar $k^* = 17^*$



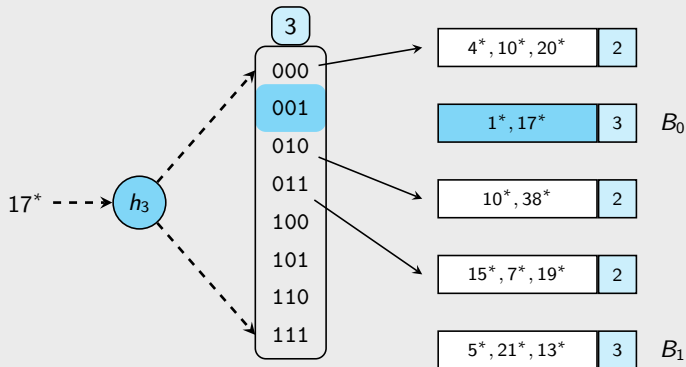
Insertar en Extendable Hashing: doblamiento del directorio

Ejemplo: insertar $k^* = 17^*$



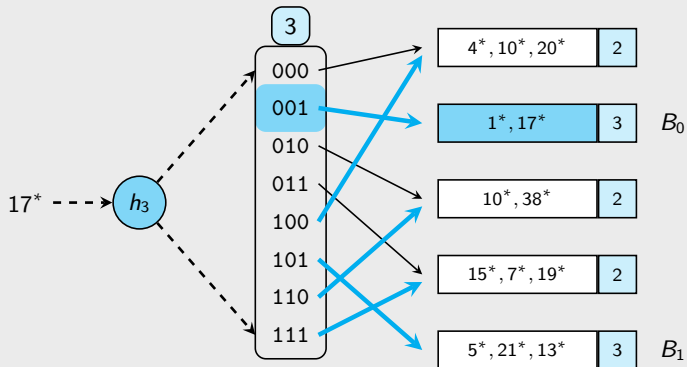
Insertar en Extendable Hashing: doblamiento del directorio

Ejemplo: insertar $k^* = 17^*$



Insertar en Extendable Hashing: doblamiento del directorio

Ejemplo: insertar $k^* = 17^*$



Eliminación en Extendable Hashing

- Para eliminar, uno busca el elemento k^* y lo elimina del bucket.
- Si el bucket **queda vacío**, extendable hashing hace **merge** de buckets.
 - ¿cuándo ocurre una disminución de la profundidad **local**?
 - ¿cuándo ocurre una disminución de la profundidad **global**?

En general, merge de bucket es **poco usado**

¿por qué?

Eficiencia de Extendable Hashing

- D : tamaño (en páginas) del directorio.

Costo de cada operación (en I/O)

Búsqueda: $\mathcal{O}(1)$

Insertar: $\mathcal{O}(D)$

Outline

Static Hashing

Extendable Hashing

Sobre índices

B+tree versus Hash

```
CREATE INDEX <nombre-indice>  
ON <nombre-rel> (<nombre-attr1>, ..., <nombre-attrk>)  
USING <index-type>;
```

$\langle \text{index-type} \rangle := \text{BTREE} \mid \text{HASH} \mid \underbrace{\dots}_{\text{depende del motor}}$

Ejemplo: MySQL

```
CREATE INDEX PlayersAgeIndex  
ON Players (age) USING BTREE;  
  
CREATE INDEX PlayersNameTeamIndex  
ON Players (name, team) USING HASH;
```

B+tree versus Hash

```
CREATE INDEX <nombre-indice>  
ON <nombre-rel> (<nombre-attr1>, ..., <nombre-attrk>)  
USING <index-type>;
```

$\langle \text{index-type} \rangle := \text{BTREE} \mid \text{HASH} \mid \underbrace{\dots}_{\text{depende del motor}}$

Ejemplo: PostgreSQL

```
CREATE INDEX PlayersAgeIndex  
ON Players USING btree (age);  
  
CREATE INDEX PlayersNameTeamIndex  
ON Players USING hash (name, team);
```

CREATE INDEX **no** es parte del estándar de SQL

(cada motor tiene su propia sintaxis)

B+tree versus Hash

B+tree

- Consultas por **valor**: =
- Consultas por **rango**: <, >, <=, >=, BETWEEN
- ORDER BY

Hash

- Consultas por **valor**: =
 - Es ligeramente más eficiente que B+tree.
- Consultas por **rango**: **NO** son soportadas.
- ORDER BY: **NO** son soportadas.

Ambos pueden **acelerar algunas consultas** como joins

(lo veremos en las próximas clases)

Clustered versus unclustered: componentes de un índice

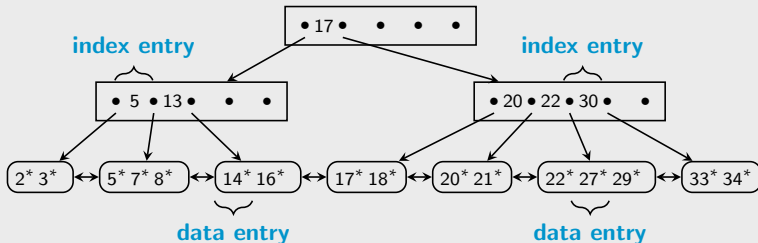
Definición

Search key := parámetro de **búsqueda**

Index entry := valor o puntero de la **estructura** de un índice

Data entry := **tupla** o **dirección** donde se almacena una tupla

Ejemplo: B+tree



Clustered versus unclustered: componentes de un índice

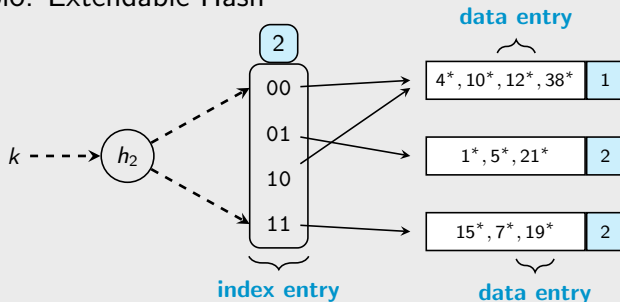
Definición

Search key := parámetro de **búsqueda**

Index entry := valor o puntero de la **estructura** de un índice

Data entry := **tupla** o **dirección** donde se almacena una tupla

Ejemplo: Extendable Hash



Clustered versus unclustered: definición

Definición

Search key := parámetro de **búsqueda**

Index entry := valor o puntero de la **estructura** de un índice

Data entry := **tupla** o **dirección** donde se almacena una tupla

Definición

Clustered index := índice para el cual el orden de sus data entries es el **mismo orden de las tuplas** en disco.

Unclustered index := índice que **NO** es clustered.

En clustered index: **data entries = tuplas**

Clustered versus unclustered: ejemplos

Definición

Clustered index := índice para el cual el orden de sus data entries es el **mismo orden de las tuplas** en disco.

Unclustered index := índice que **NO** es clustered.

¿clustered o unclustered?

Players(ID, name, number)

A		232	Alexander-Arnold	12
H		520	Bellingham	5
M		335	Halland	9
		481	Kane	9
		630	Mbappé	10
		555	Salah	10

Clustered versus unclustered: ejemplos

Definición

Clustered index := índice para el cual el orden de sus data entries es el **mismo orden de las tuplas** en disco.

Unclustered index := índice que **NO** es clustered.

¿clustered o unclustered?

Players(ID, name, number)

A		Alexander-Arnold	1C9AB2
H		Bellingham	74C6B7
M		Halland	C56F79
		Kane	AA45B6
		Mbappé	426FF4
		Salah	1265FA

Clustered versus unclustered: comentarios

- En general, por cada relación es posible mantener **un solo índice clustered**. (¿por que?)
- Índice unclustered son **ineficientes** cuando el output es numeroso.
 - Ejemplo: consultas por rango
- Usualmente, clustered y unclustered son conocidos como:
 - clustered index = índice **primario**.
 - unclustered index = índice **secundario**.

Usualmente, índice **clustered** esta reservado para la PRIMARY KEY

(depende del motor de BD)